

VEHICLE PARKING AND TICKET MANAGEMENT SYSTEM

1. Project Title

Vehicle Parking and Ticket Management System

2. Project Outcome

The Vehicle Parking and Ticket Management System was successfully developed using Spring Boot and MongoDB. The project automates parking operations by managing drivers, vehicles, parking slots, tickets, and payments through REST APIs.

Achievements:

- Implemented complete CRUD operations.
- Managed driver and vehicle records efficiently.
- Established Many-to-Many relationship between Drivers and Vehicles.
- Automated parking slot allocation.
- Generated parking tickets for vehicle entry.
- Recorded vehicle exits.
- Managed payment status and payment records.

HTTP Methods Used in REST APIs

GET Method:

The GET method is used to retrieve or fetch data from the server. It does not modify any existing data and is mainly used for viewing records. For example, fetching all drivers, vehicles, parking slots, or tickets from the database.

POST Method:

The POST method is used to create and store new data on the server. It sends information from the client to the server for insertion into the database. For example, registering a new vehicle, adding a driver, generating a parking ticket, or recording vehicle entry.

PUT Method:

The PUT method is used to update existing records in the database. It modifies the information of a resource that already exists. For example, updating driver details, changing vehicle information, or updating the status of a parking slot.

DELETE Method:

The DELETE method is used to remove data from the server permanently. It deletes specific records from the database. For example, deleting a driver, vehicle, parking slot, ticket, or payment record.

These HTTP methods form the foundation of CRUD operations:

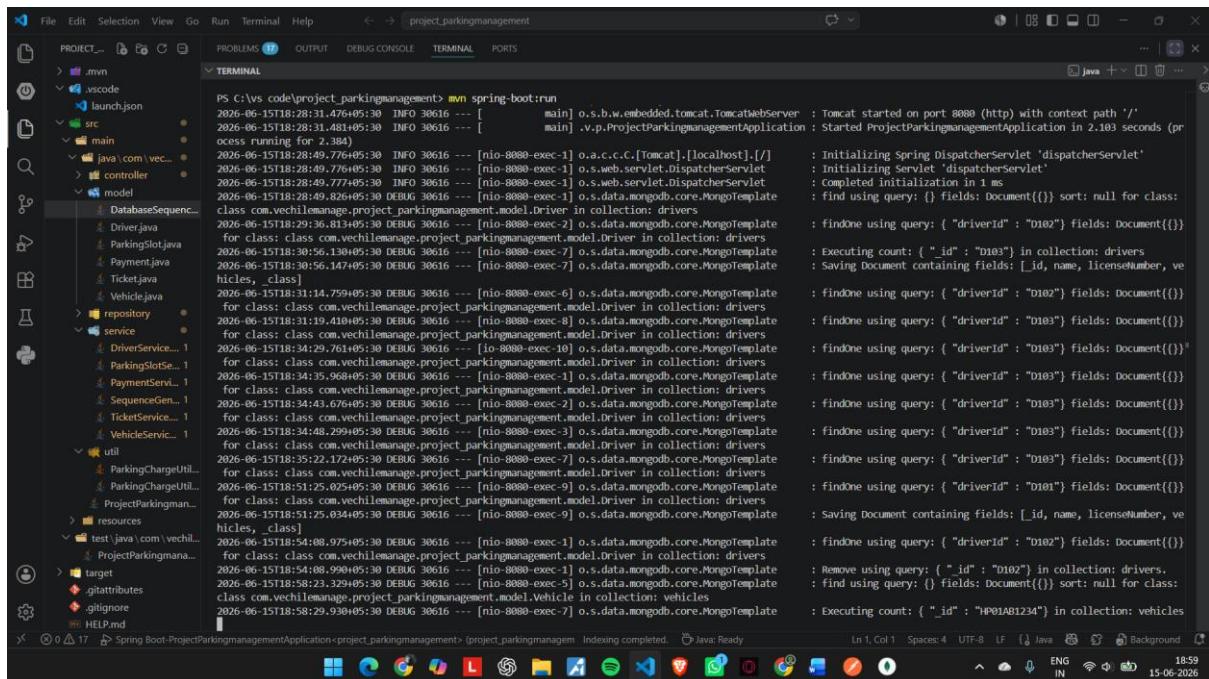
- **Create → POST**
- **Read → GET**
- **Update → PUT**
- **Delete → DELETE**

3. Tools Used

Tool	Purpose	Version
Java	Programming Language	JDK 25
Spring Boot	Backend Framework	3.5.3
MongoDB Compass	Database GUI	Latest
Postman	API Testing	Latest
VS Code	Development Environment	Latest

4. Output Screenshots with Explanation

Screenshot 1: Application Execution in VS Code

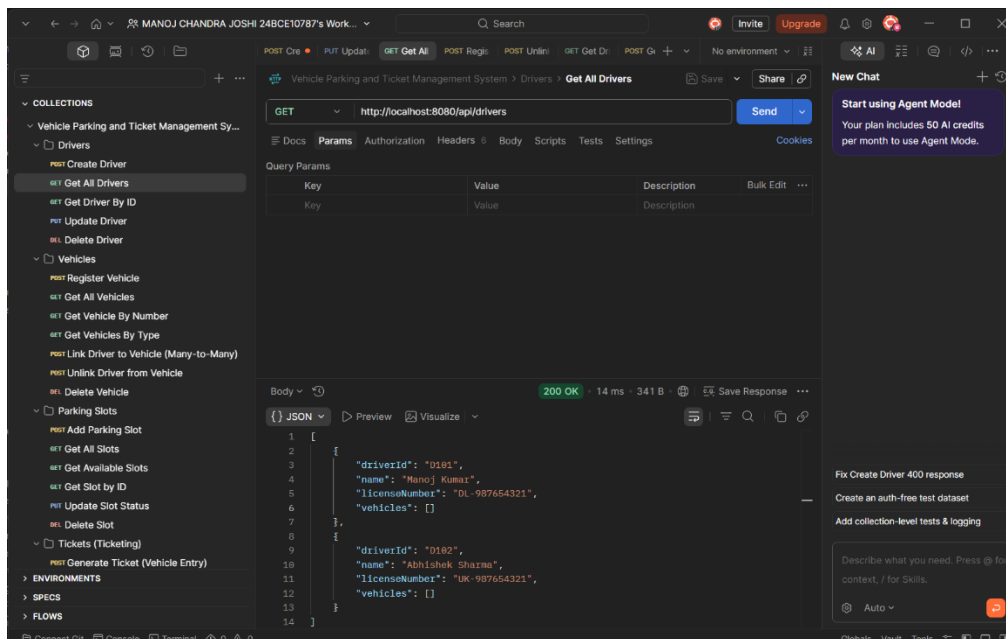


```
PS C:\vs_code\project_parkingmanagement> mvn spring-boot:run
2026-06-15T18:28:31.476405:30 INFO 30616 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-06-15T18:28:31.481405:30 INFO 30616 --- [main] v.p.ProjectParkingmanagementApplication : Started ProjectParkingmanagementApplication in 2.103 seconds (pr
cess running for 2.384)
2026-06-15T18:28:49.776405:30 INFO 30616 --- [nio-8080-exec-1] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-15T18:28:49.777405:30 INFO 30616 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-15T18:28:49.826405:30 DEBUG 30616 --- [nio-8080-exec-1] o.s.data.mongodb.core.MongoTemplate : Completed initialization in 1 ms
2026-06-15T18:29:36.813405:30 DEBUG 30616 --- [nio-8080-exec-2] o.s.data.mongodb.core.MongoTemplate : find using query: {} fields: Document({}) sort: null for class:
class com.vechilemanage.project_parkingmanagement.model.Driver in collection: drivers
2026-06-15T18:29:36.813405:30 DEBUG 30616 --- [nio-8080-exec-2] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "_id": "D102" } fields: Document({})
2026-06-15T18:30:56.130405:30 DEBUG 30616 --- [nio-8080-exec-7] o.s.data.mongodb.core.MongoTemplate : Executing count: { "_id": "D103" } in collection: drivers
2026-06-15T18:30:56.147405:30 DEBUG 30616 --- [nio-8080-exec-7] o.s.data.mongodb.core.MongoTemplate : Saving Document containing fields: [_id, name, licenseNumber, ve
hicles, _class]
2026-06-15T18:31:14.759405:30 DEBUG 30616 --- [nio-8080-exec-6] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D102" } fields: Document({})
2026-06-15T18:31:19.410405:30 DEBUG 30616 --- [nio-8080-exec-8] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:34:29.761405:30 DEBUG 30616 --- [io-8080-exec-10] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:34:35.968405:30 DEBUG 30616 --- [nio-8080-exec-1] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:34:42.676405:30 DEBUG 30616 --- [nio-8080-exec-2] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:34:48.299405:30 DEBUG 30616 --- [nio-8080-exec-3] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:35:22.172405:30 DEBUG 30616 --- [nio-8080-exec-7] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D103" } fields: Document({})
2026-06-15T18:51:25.025405:30 DEBUG 30616 --- [nio-8080-exec-9] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D101" } fields: Document({})
2026-06-15T18:51:25.034405:30 DEBUG 30616 --- [nio-8080-exec-9] o.s.data.mongodb.core.MongoTemplate : Saving Document containing fields: [_id, name, licenseNumber, ve
hicles, _class]
2026-06-15T18:54:08.975405:30 DEBUG 30616 --- [nio-8080-exec-1] o.s.data.mongodb.core.MongoTemplate : findOne using query: { "driverId": "D102" } fields: Document({})
2026-06-15T18:54:08.990405:30 DEBUG 30616 --- [nio-8080-exec-1] o.s.data.mongodb.core.MongoTemplate : Remove using query: { "_id": "D102" } in collection: drivers.
2026-06-15T18:58:23.329405:30 DEBUG 30616 --- [nio-8080-exec-5] o.s.data.mongodb.core.MongoTemplate : find using query: {} fields: Document({}) sort: null for class:
class com.vechilemanage.project_parkingmanagement.model.Vehicle in collection: vehicles
2026-06-15T18:58:29.930405:30 DEBUG 30616 --- [nio-8080-exec-7] o.s.data.mongodb.core.MongoTemplate : Executing count: { "_id": "H01A81234" } in collection: vehicles
```

Explanation:

The Spring Boot application is successfully running in VS Code terminal. API requests sent from Postman are processed and reflected in the terminal logs.

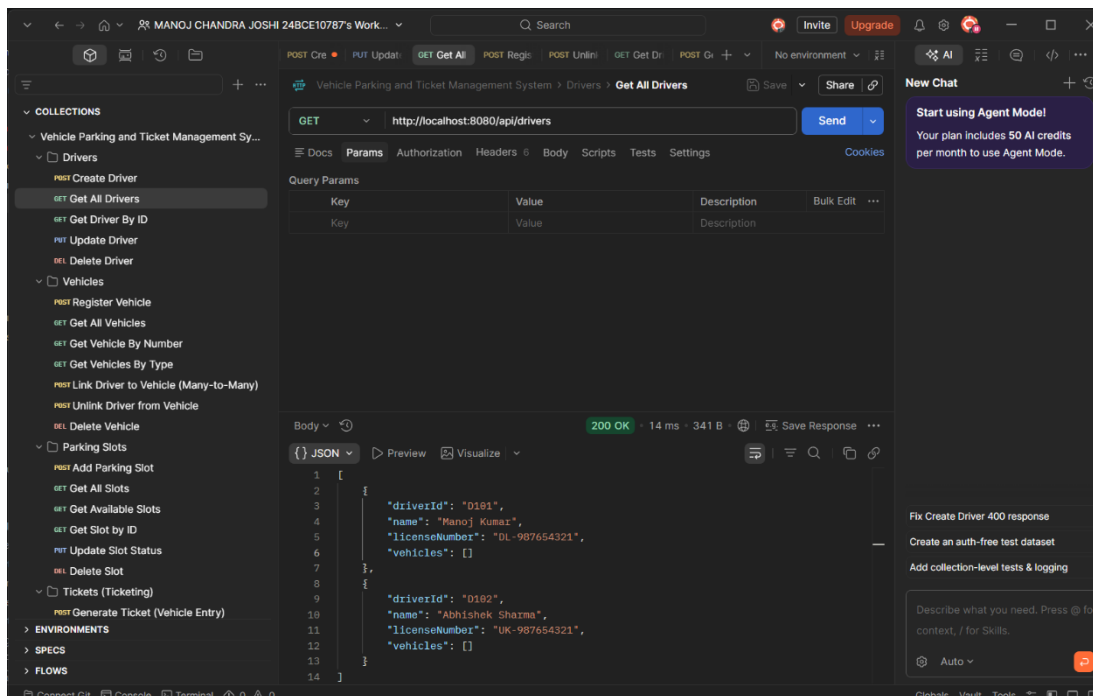
Screenshot 2: Create Driver (POST)



Explanation:

A new driver record is created using the POST API endpoint and stored in MongoDB.

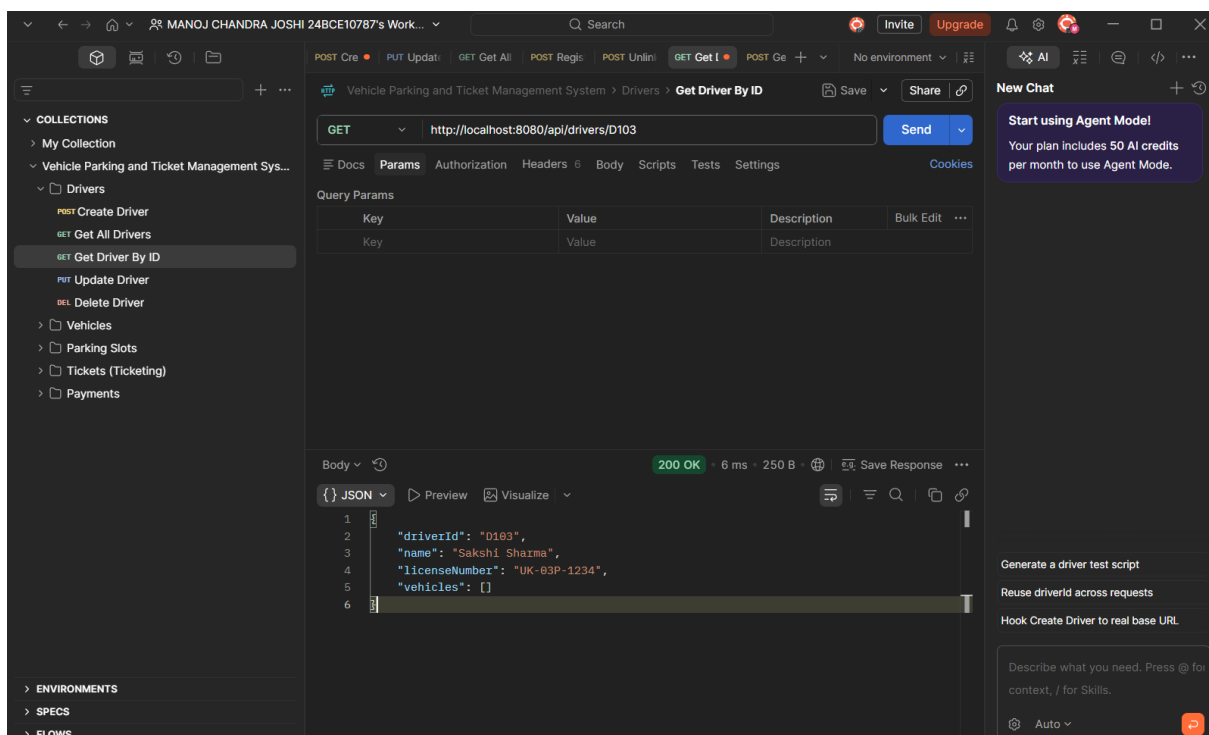
Screenshot 3: Get All Drivers (GET)



Explanation:

The GET API retrieves all driver records from the database.

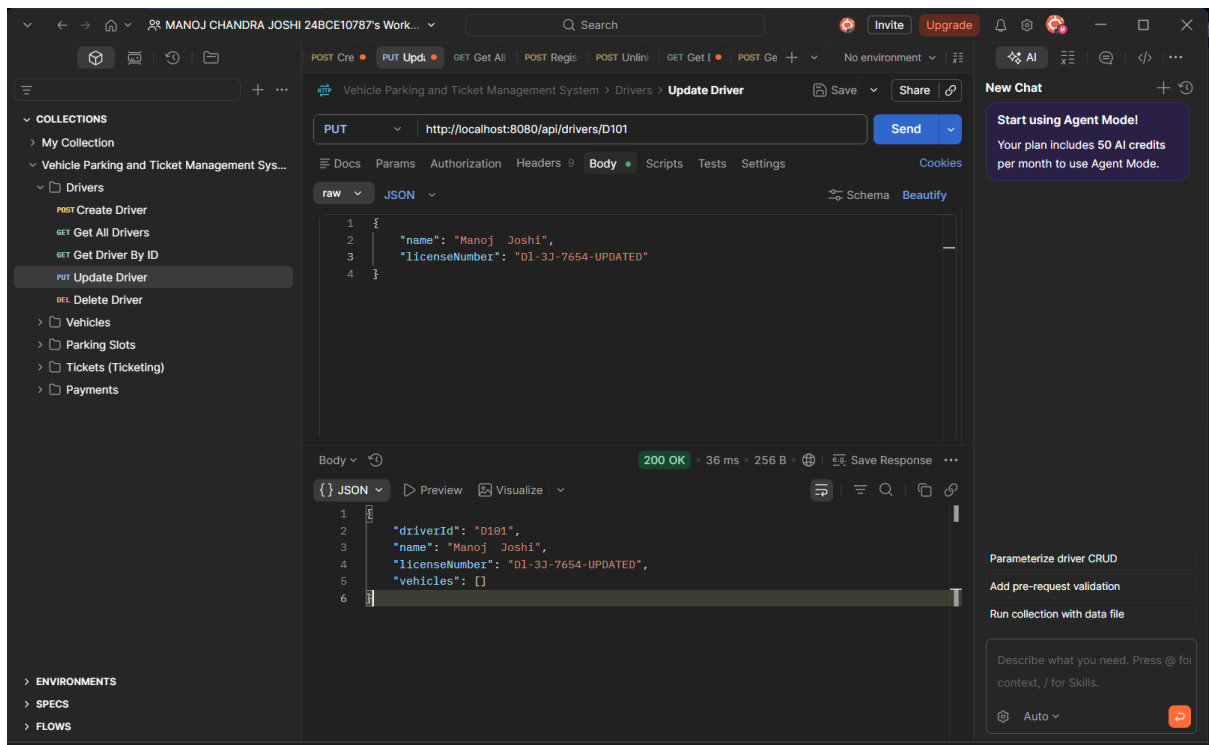
Screenshot 4: Get Driver By ID



Explanation:

Specific driver information is fetched using the driver ID.

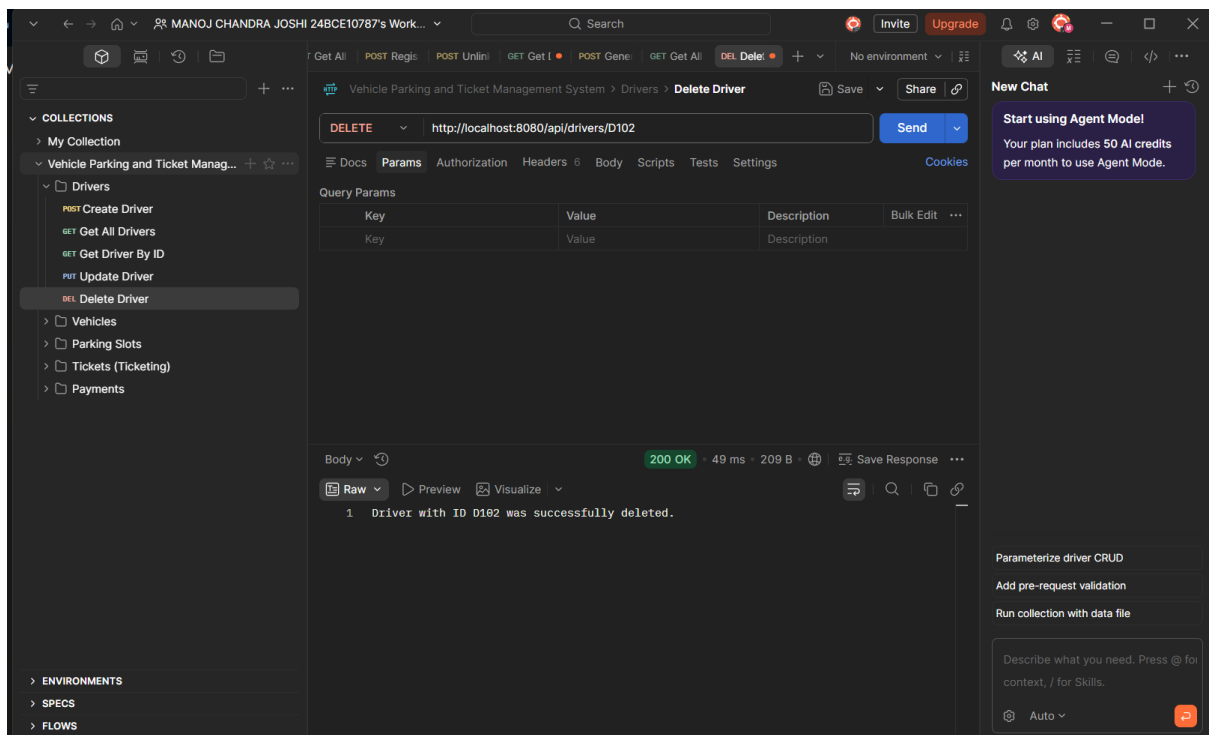
Screenshot 5: Update Driver (PUT)



Explanation:

Driver details such as name and license number are updated successfully.

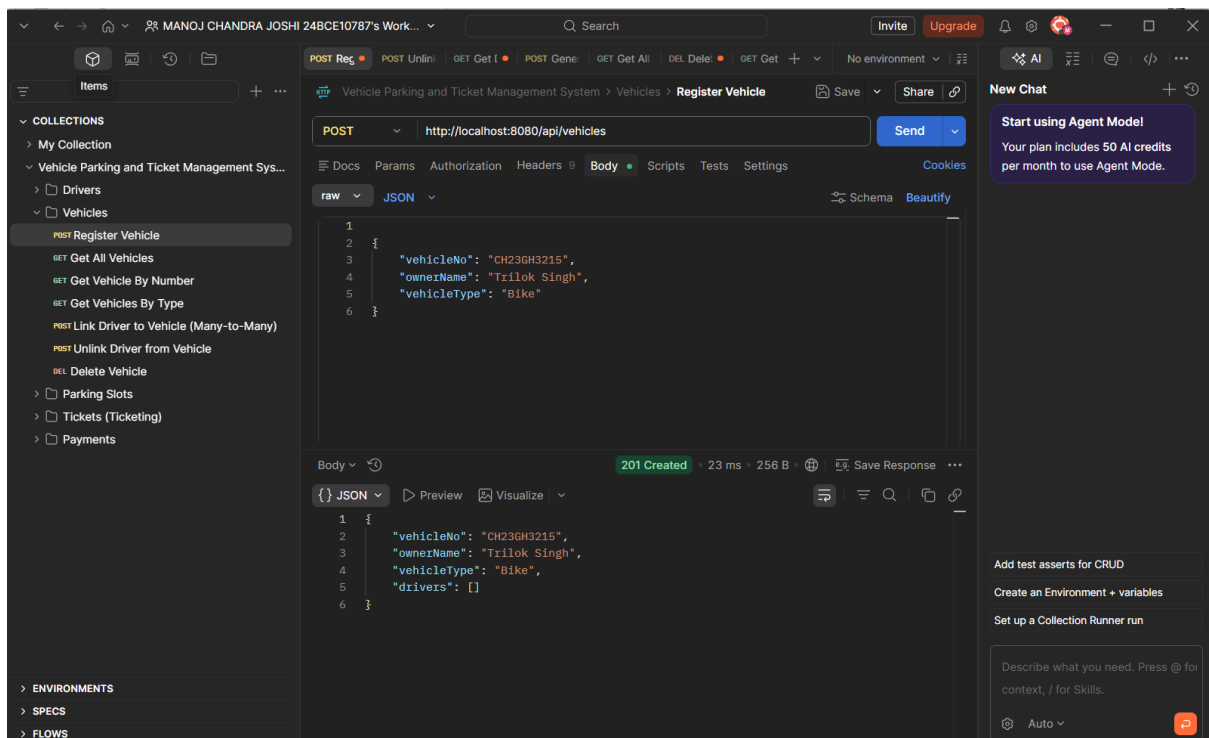
Screenshot 6: Delete Driver (DELETE)



Explanation:

The selected driver record is removed from the database.

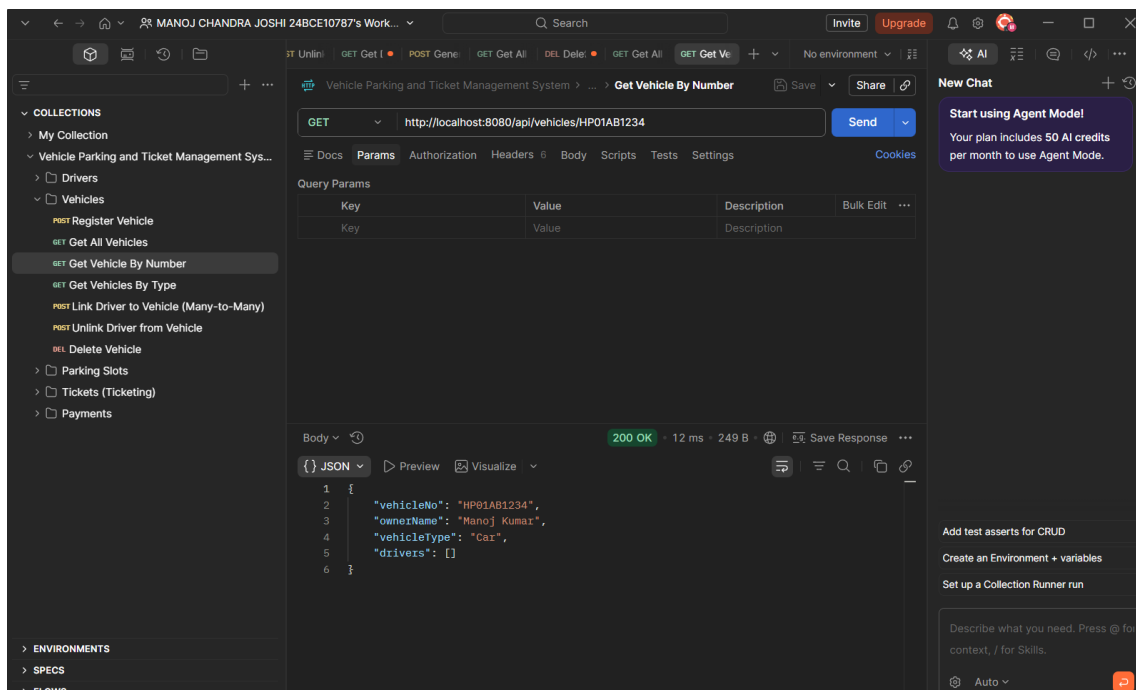
Screenshot 7: Vehicle Registration



Explanation:

A new vehicle is registered in the system. Duplicate vehicle numbers are restricted to maintain data integrity.

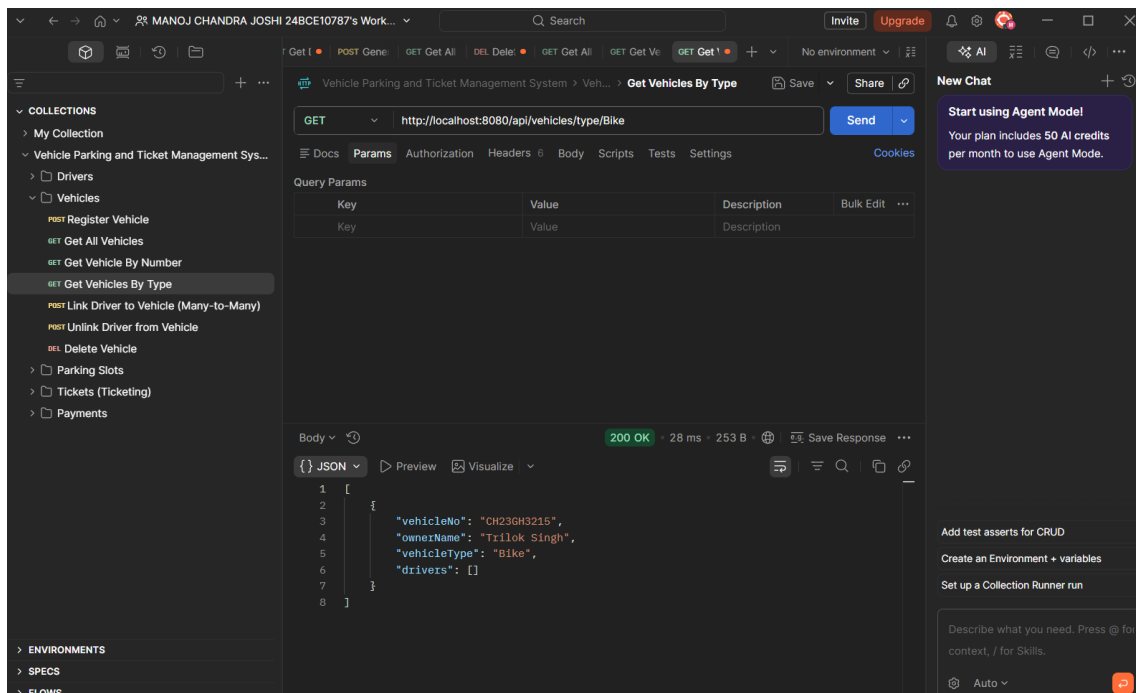
Screenshot 8: Get Vehicle By Number



Explanation:

Vehicle information is retrieved using the vehicle number.

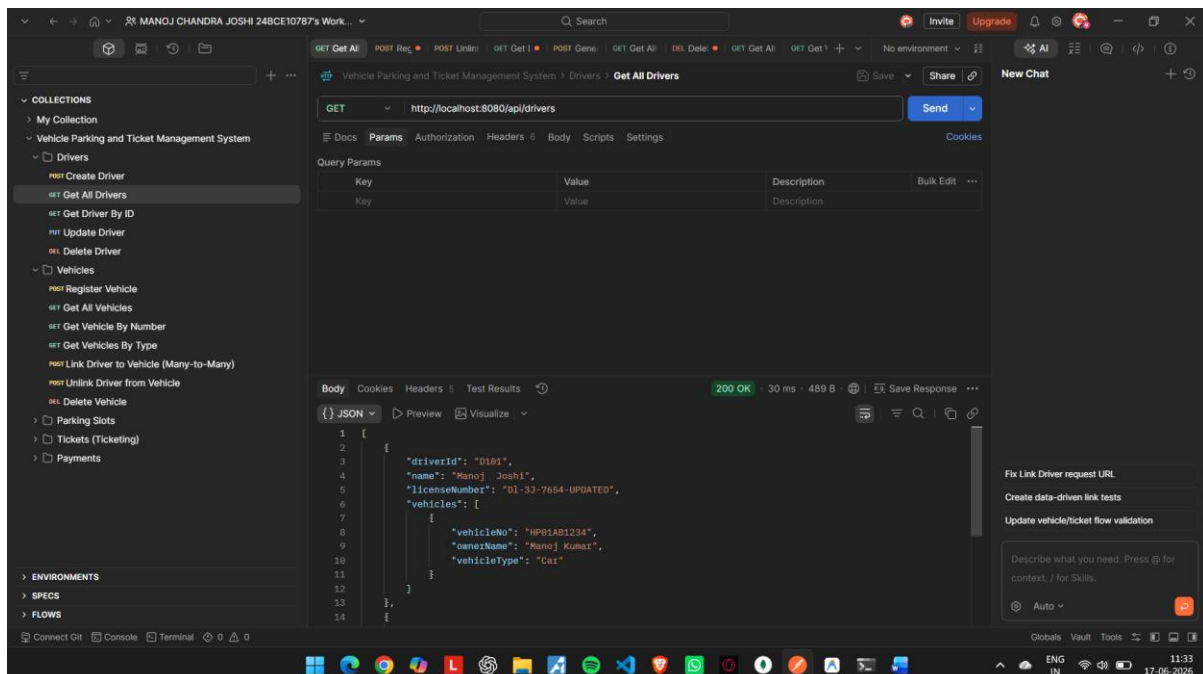
Screenshot 9: Get Vehicle By Type



Explanation:

Vehicles can be filtered and displayed according to their type such as Car or Bike.

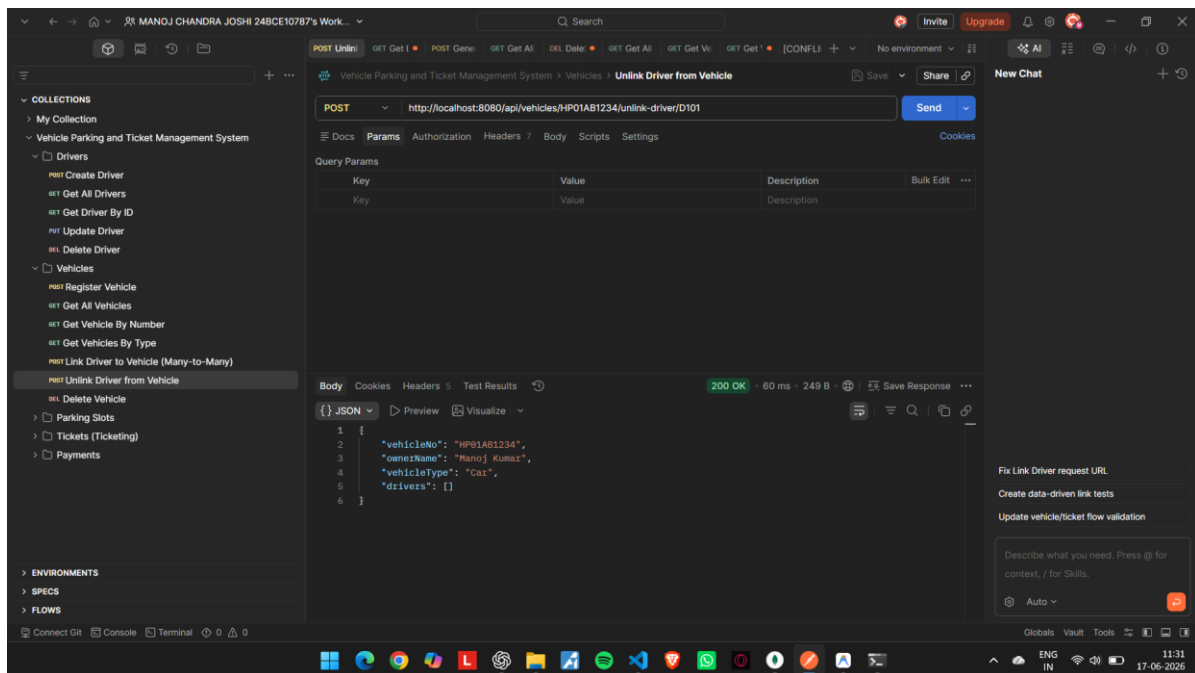
Screenshot 10: Driver-Vehicle Linking



Explanation:

A Many-to-Many relationship is established between drivers and vehicles.

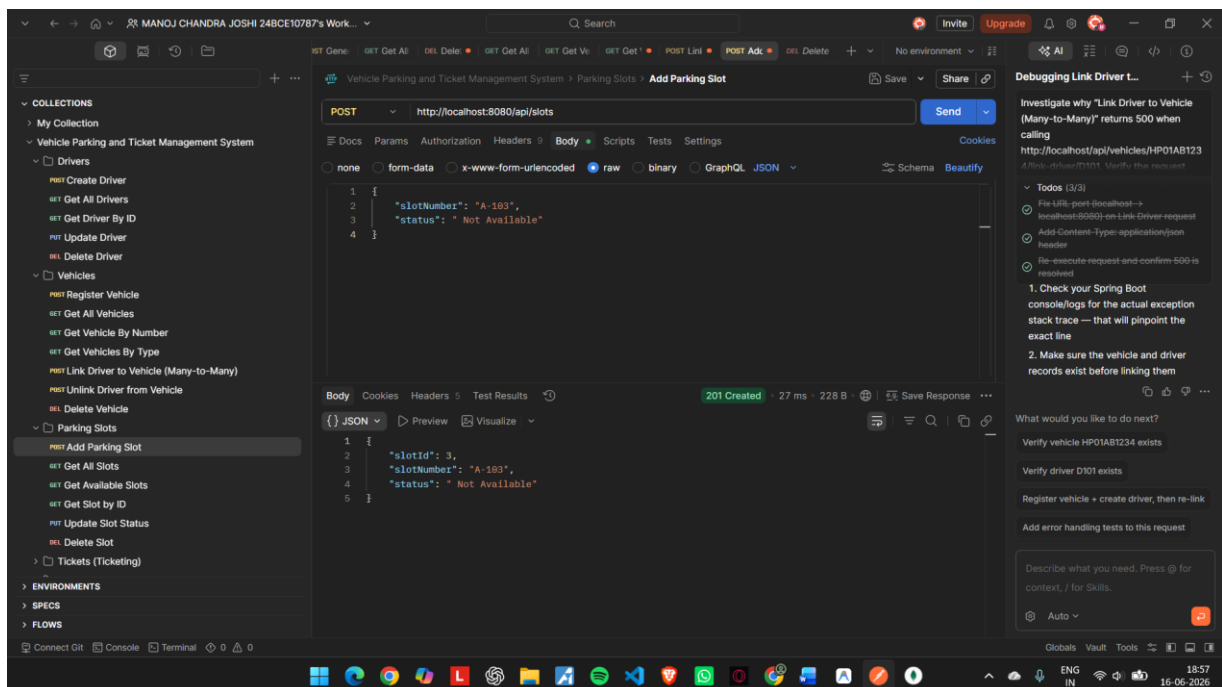
Screenshot 11 : Driver-Vehicle Unlinking



Explanation:

Previously linked driver and vehicle records can be separated.

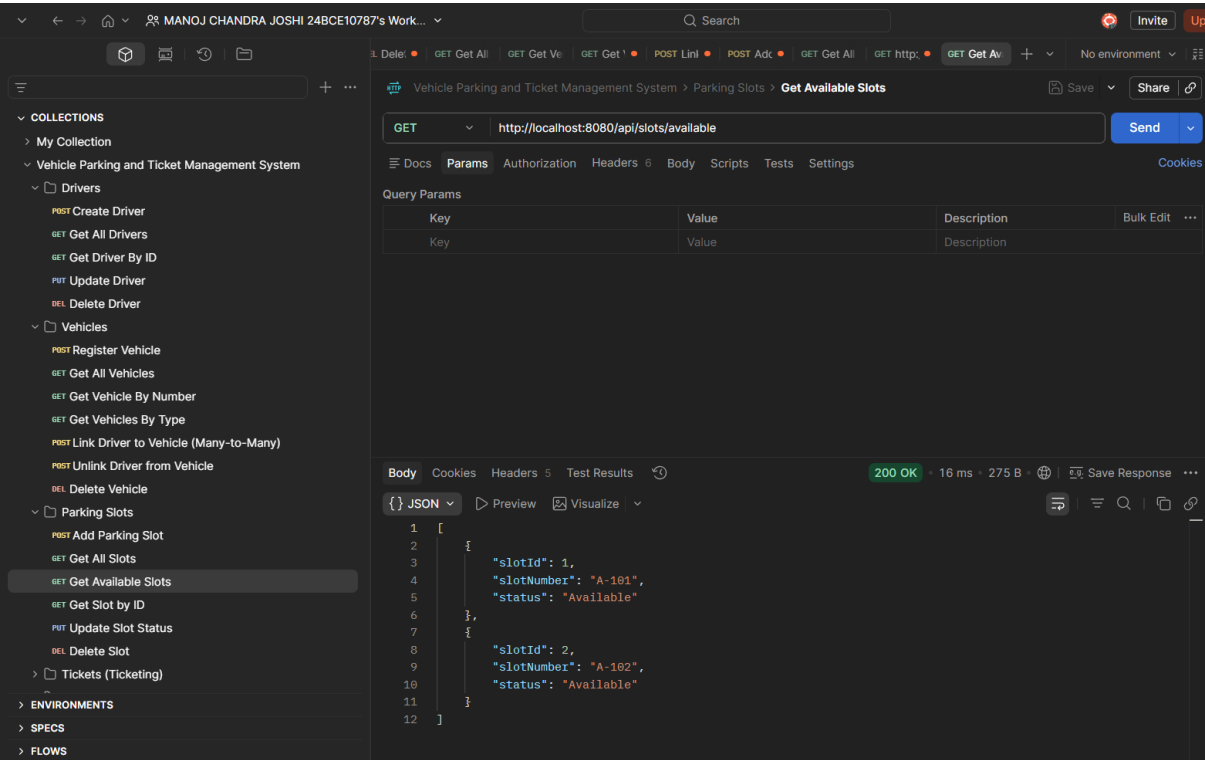
Screenshot 12: Add Parking Slot



Explanation:

New parking slots are created with unique slot numbers.

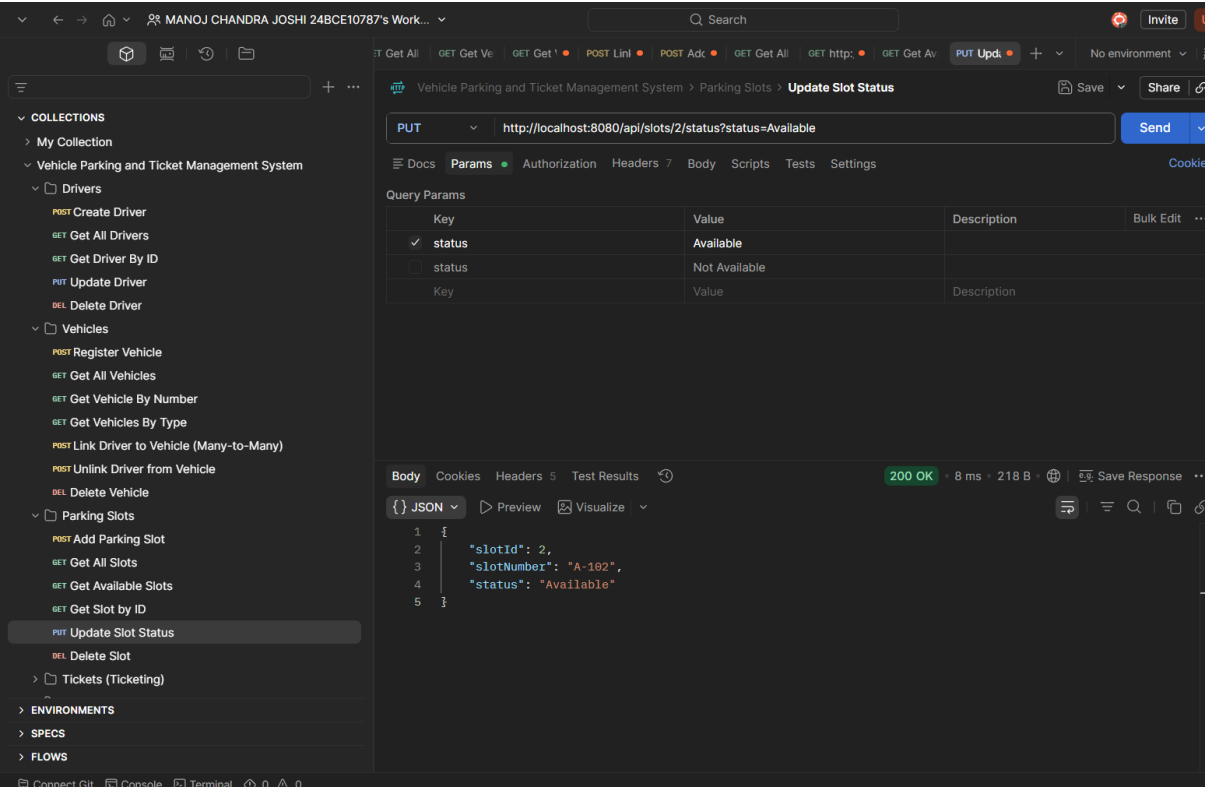
Screenshot 13: Get Available Parking Slots



Explanation:

The system displays all parking slots currently available for vehicle parking.

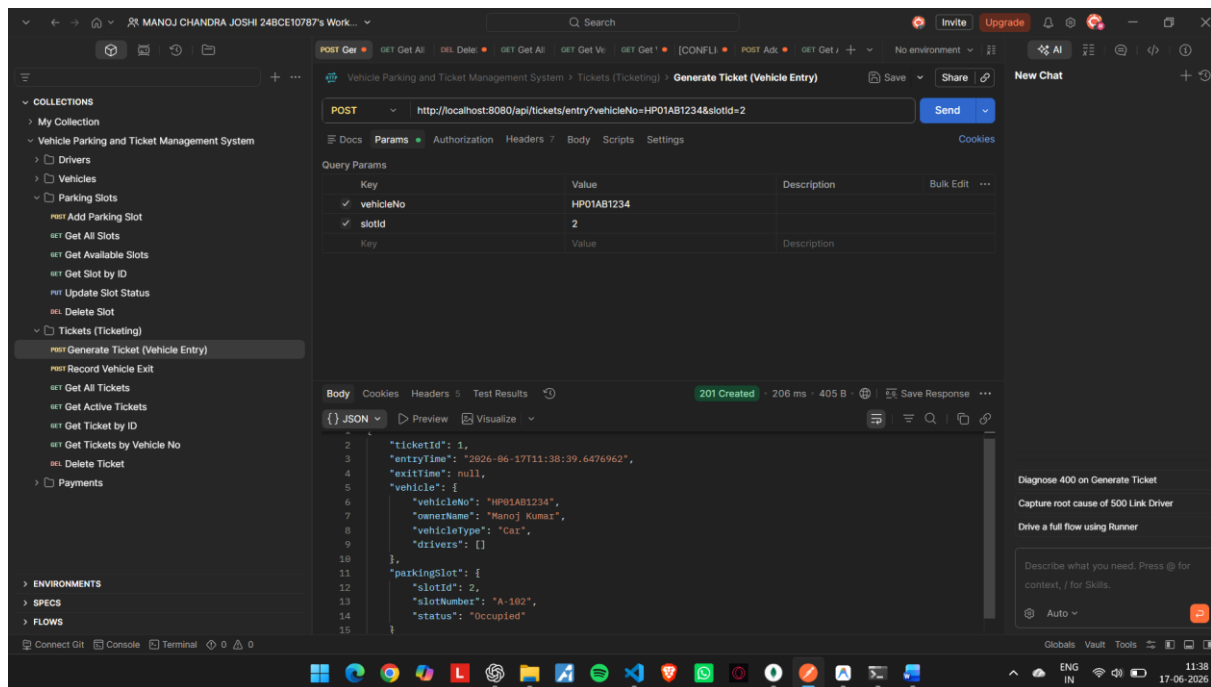
Screenshot 14: Update Parking Slot Status



Explanation:

Parking slot status changes automatically between Available and Not Available.

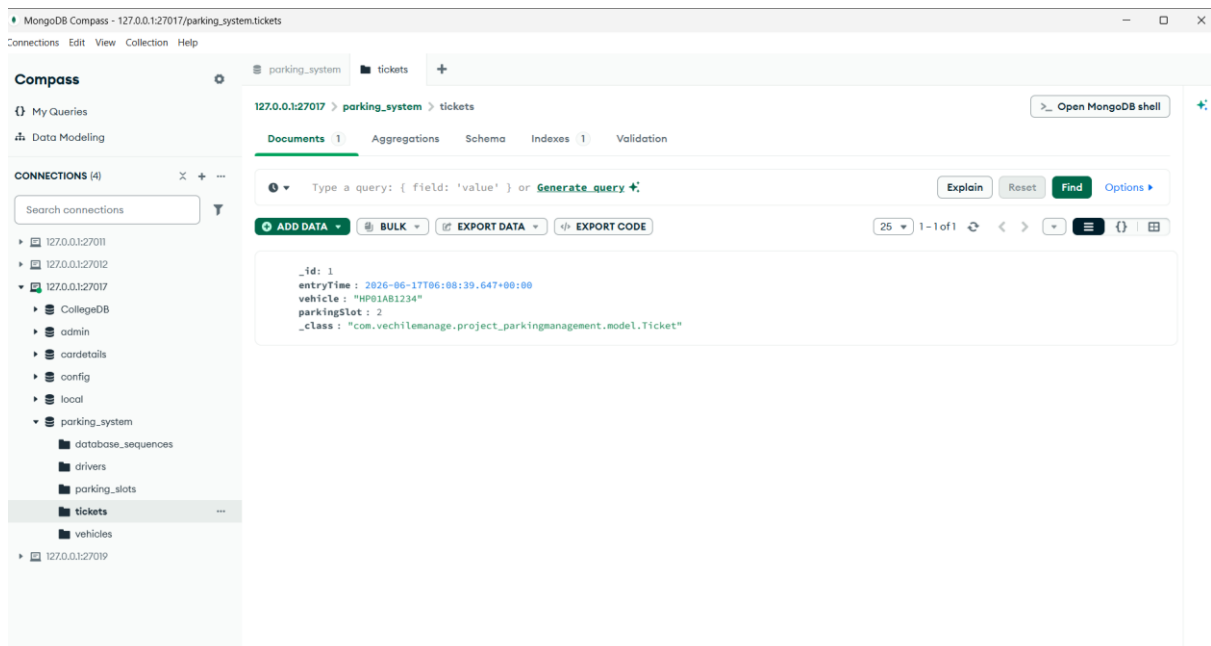
Screenshot 15: Generate Ticket



Explanation:

When a vehicle enters the parking area, a parking ticket is generated automatically.

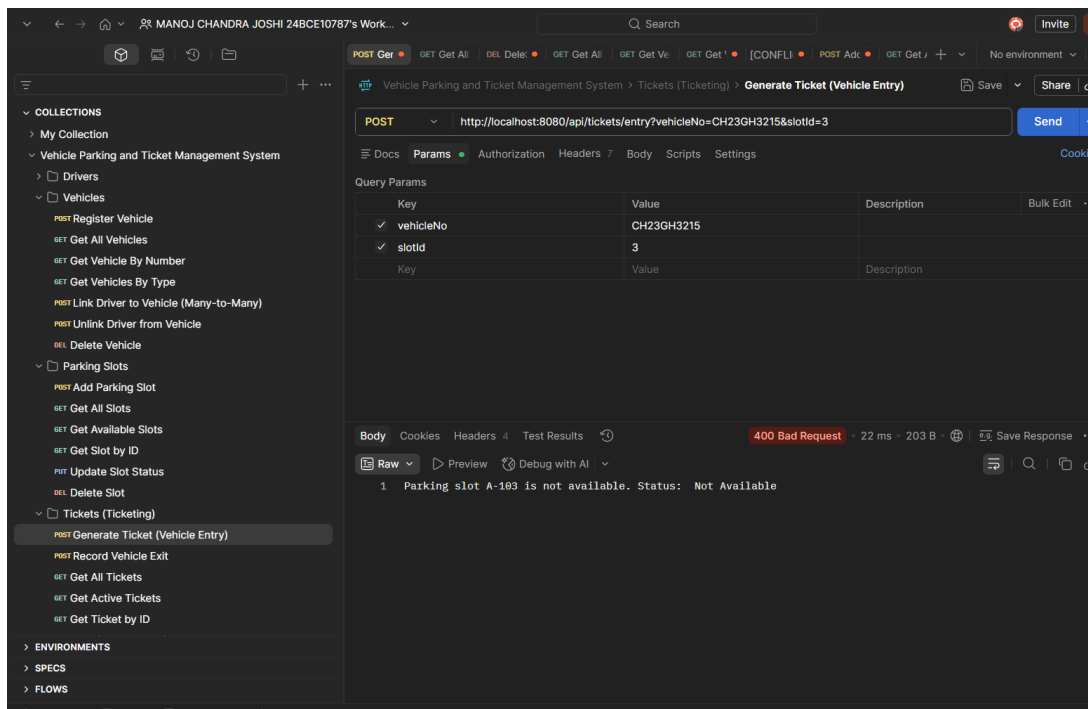
Screenshot 16: Ticket Database



Explanation:

Generated ticket information is stored in the MongoDB tickets collection.

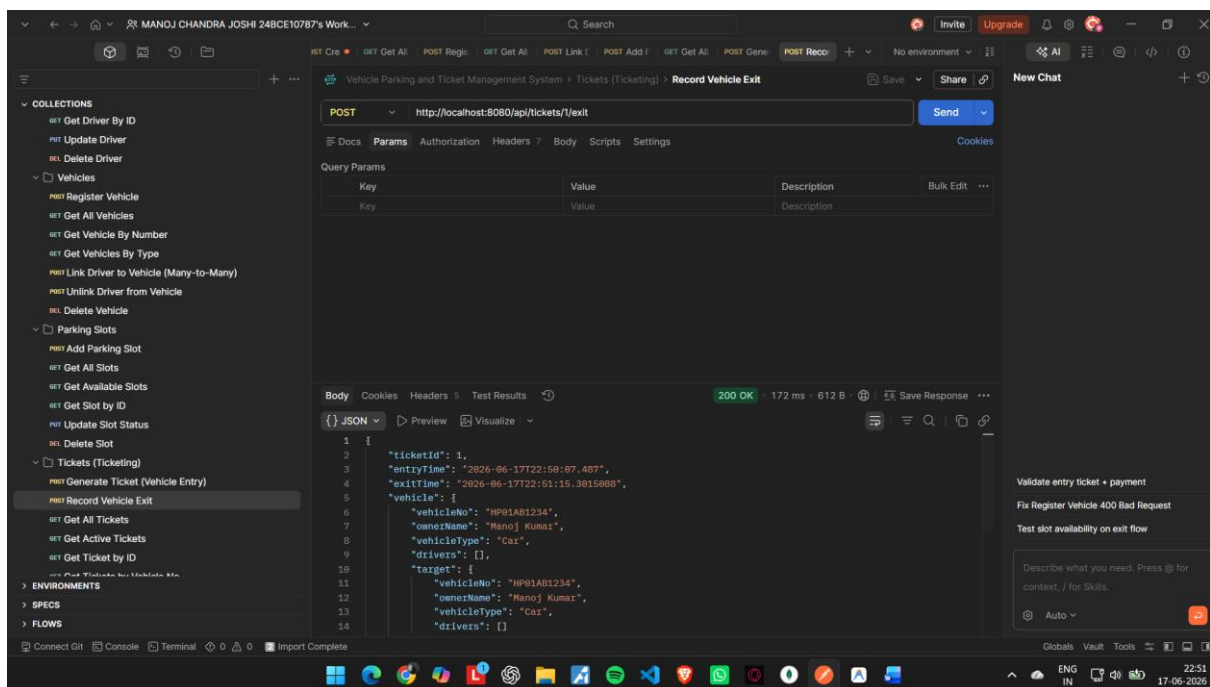
Screenshot 17: Parking Slot Validation



Explanation:

The system prevents ticket generation for occupied slots and returns a 400 Bad Request response.

Screenshot 18: Record Vehicle Exit



Explanation:

Vehicle exit details are recorded, updating the corresponding ticket.

Screenshot 19: Get All Tickets

The screenshot shows the Postman interface for the 'Get All Tickets' endpoint. The URL is `http://localhost:8080/api/tickets`. The response is a 200 OK status with a JSON array containing one ticket object.

```
{
  "ticketId": 1,
  "entryTime": "2026-06-17T11:38:39.647",
  "exitTime": null,
  "vehicle": {
    "vehicleNo": "HP01AB1234",
    "ownerName": "Manoj Kumar",
    "vehicleType": "Car",
    "drivers": [],
    "target": {
      "vehicleNo": "HP01AB1234",
      "ownerName": "Manoj Kumar",
      "vehicleType": "Car",
    }
  }
}
```

Explanation:

Displays all active and completed parking tickets.

Screenshot 20: Get Ticket By ID

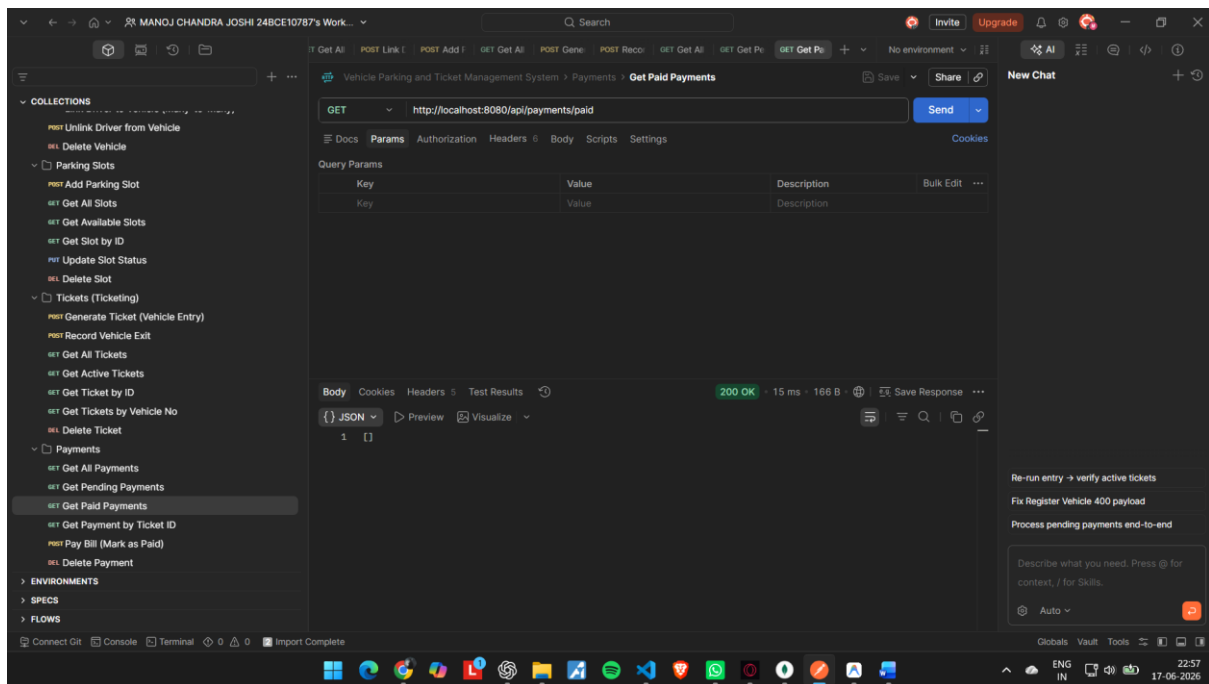
The screenshot shows the Postman interface for the 'Get Tickets by Vehicle No' endpoint. The URL is `http://localhost:8080/api/tickets/vehicle/UK-05T-5674`. The response is a 200 OK status with a JSON object containing ticket details.

```
{
  "ticketId": 2,
  "entryTime": "2026-06-17T11:51:36.626",
  "exitTime": null,
  "vehicle": {
    "vehicleNo": "UK-05T-5674",
    "ownerName": "ANUBHAV SINGH",
    "vehicleType": "Bike",
    "drivers": [],
    "target": {
      "vehicleNo": "UK-05T-5674",
      "ownerName": "ANUBHAV SINGH",
      "vehicleType": "Bike",
    }
  }
}
```

Explanation:

Retrieves a specific ticket using its ticket ID.

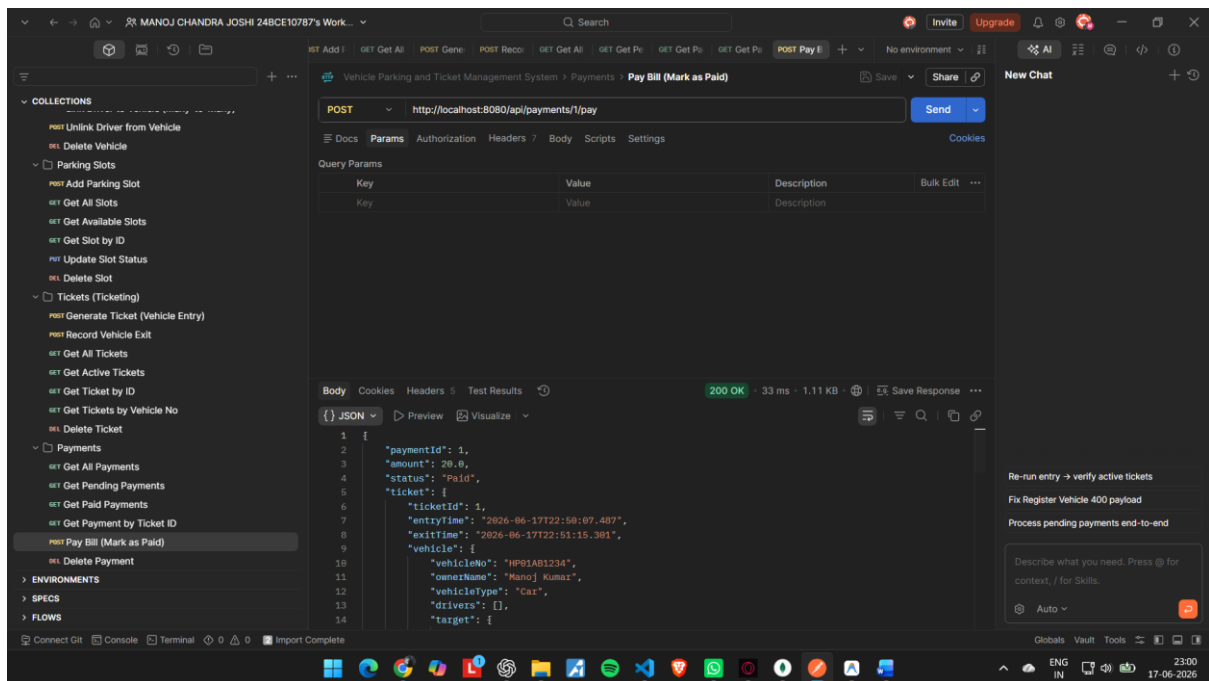
Screenshot 21: Payment Management



Explanation:

The system manages pending and completed payments linked to parking tickets.

Screenshot 22: Paid Payment Status



Explanation:

After successful payment, the payment status changes from Pending to Paid.

5. Project Summary

The Vehicle Parking and Ticket Management System is a backend application developed using Java Spring Boot and MongoDB. The system manages parking operations through REST APIs and supports driver management, vehicle registration, parking slot management, ticket generation, vehicle exit tracking, and payment processing. MongoDB is used as the database, while Postman is used for API testing. The project demonstrates database integration, CRUD operations, relationship mapping, and real-time parking management functionalities.

6. Conclusion

The Vehicle Parking and Ticket Management System successfully automates the parking workflow and minimizes manual effort. The project demonstrates the practical implementation of Spring Boot REST APIs with MongoDB integration. Features such as vehicle registration, parking slot allocation, ticket generation, and payment management improve efficiency and accuracy. The project provides a strong foundation for future enhancements such as online payments, QR-based tickets, and frontend integration.